

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО «СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
ИНСТИТУТ ЦИФРОВОГО РАЗВИТИЯ
МЕЖИНСТИТУТСКАЯ БАЗОВАЯ КАФЕДРА

КУРСОВОЙ ПРОЕКТ

по дисциплине

«Технология разработки программного обеспечения»

на тему:

«Разработка веб-приложения клавиатурного онлайн-тренажера»

Выполнил:

Усенко Николай Игоревич студент 2
курса группы ПИЖ-б-о-24-1
направления подготовки 09.03.04
«Программная инженерия»
направленность (профиль)
«Разработка и сопровождение
программного обеспечения»
очной формы обучения

(подпись)

Руководитель работы:
Ф.В. Самойлов, доцент
межинститутской базовой кафедры

Работа допущена к защите

(подпись руководителя)

(дата)

Работа выполнена и
защищена с оценкой

Дата защиты

Председатель комиссии:
зав. межинститутской
базовой кафедрой

(подпись)

Е. Н. Новикова

Члены комиссии:
доцент МИБК

(подпись)

Ф.В. Самойлов

доцент МИБК

(подпись)

И.В. Свистунов

Ставрополь, 2026 г.

Министерство науки и высшего образования и Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Кафедра межинститутская базовая

Направление подготовки 09.03.04 «Программная инженерия»

Направленность (профиль) «Разработка и сопровождение программного обеспечения»

ЗАДАНИЕ

на курсовой проект

студента Усенко Николая Игоревича

(фамилия, имя, отчество)

по дисциплине «Технология разработки программного обеспечения»

1. Тема проекта Разработка веб-приложения клавиатурного онлайн-тренажера
2. Цель: Повышение уровня профессиональной подготовки путем углубления и закрепления теоретических знаний и практических навыков, приобретенных в результате изучения дисциплины «Технология разработки программного обеспечения», а также формирование компетенций в области проектирования, разработки и документирования современных веб-приложений с использованием фреймворка Django в рамках выбранной траектории выполнения.
3. Задачи:
 - 3.1. Анализ прикладной задачи и методов её решения
 - Провести предпроектное обследование предметной области.
 - Выполнить анализ бизнес-процессов и сформулировать требования к программному обеспечению.
 - Обосновать выбор технологического стека в соответствии с выбранной траекторией:
 - Траектория В: Django, Django REST Framework, React, Django Channels, WebSocket, JWT, PostgreSQL, Redis.
 - Проанализировать существующие аналоги и обосновать необходимость разработки.
 - 3.2. Разработка алгоритмов решения задачи
 - Разработать модель предметной области (Domain Model).
 - Спроектировать Use Case диаграмму и спецификации прецедентов.
 - Разработать ER-диаграмму базы данных.

- Спроектировать архитектуру приложения в соответствии с выбранной траекторией:
- Траектория В: клиент-серверная архитектура с WebSocket.
- Разработать алгоритмы основных сценариев (регистрация, аутентификация, CRUD-операции).

3.3. Реализация программного кода

- Реализовать модели данных с учётом связей между таблицами.
- Реализовать систему аутентификации:
- Траектория В: JWT-аутентификация (access/refresh токены).
- Реализовать бизнес-логику:
- Траектория В: REST API (ViewSet, Serializer) и React-приложение.
- Реализовать дополнительные функции согласно выбранной траектории:
- Траектория В: WebSocket (Django Channels) для real-time уведомлений, оптимистичные обновления, кэширование.

3.4. Отладка и тестирование программного кода

- Провести модульное тестирование ключевых компонентов (модели, API, компоненты React).
- Выполнить интеграционное тестирование взаимодействия между слоями.
- Провести системное тестирование сценариев использования.
- Для траектории В: выполнить нагрузочное тестирование WebSocket-соединений.
- Зафиксировать результаты тестирования в отчёте.

4. Перечень подлежащих разработке вопросов:

- а) теоретической части: изучение и анализ литературы, постановка условия задачи, выбор и описание методов и библиотек для ее решения и технологий, среды программирования
- б) проектная часть: проектирование UML-диаграмм классов, разработка алгоритмов решения задачи, методов классов
- в) реализация: описание структуры проекта, описание файлов проекта, разработка программного кода, тестирование программы

5. Исходные данные:

- а) по литературным источникам: ГОСТы, международные стандарты, программные средства, используемые при разработке программного обеспечения
- б) исходные данные, подготовленные для тестирования объекта профессиональной деятельности (информационной системы / приложения / программного продукта)

6. Список рекомендуемой литературы: монографии, диссертации, научные статьи, учебно-методические материалы, ссылки на официальные сайты,

содержащие информацию необходимую для решения поставленных в работе задач

1. Кузьмин, А. Г. Разработка фронтенд-приложений / А. Г. Кузьмин. — Санкт-Петербург : Питер, 2025. — 496 с. — (Библиотека программиста). — ISBN 978-5-4461-4272-9.
2. Кухтин, С. А. Frontend-разработка сайтов и приложений на HTML, CSS, JavaScript и React / С. А. Кухтин. — Санкт-Петербург : Наука и Техника, 2026. — 512 с. — ISBN 978-5-907592-84-1.
3. Орбайсета, А. С. Создание фронтенд-фреймворка с нуля / А. С. Орбайсета. — Санкт-Петербург : Питер, 2025. — 384 с. — (Библиотека программиста). — ISBN 978-5-4461-4201-9.
4. Курс «Фронтенд-разработчик» от Яндекс Практикум. — URL: <https://practicum.yandex.ru/frontend-developer/>.
5. Проектирование и разработка WEB-приложений. Введение в frontend и backend разработку на JavaScript и node.js : учебное пособие для вузов. — 4-е изд., стер. — Санкт-Петербург : Лань, 2025. — 120 с. — ISBN 978-5-507-51011-5.
6. Чернышев, С. А. Основы Flutter / С. А. Чернышев, Ю. М. Петров, С. П. Ильин, П. А. Гершевич. — Санкт-Петербург : Питер, 2026. — 688 с. — (Библиотека программиста). — ISBN 978-5-4461-4469-3.
7. Дронов, В. А. Django 4: практика создания веб-сайтов / В. А. Дронов. — Санкт-Петербург : БХВ-Петербург, 2024. — 544 с. — ISBN 978-5-9775-6892-6.
8. Ломов, А. Ю. Веб-разработка на Django 5 / А. Ю. Ломов. — Москва : ДМК Пресс, 2025. — 412 с. — ISBN 978-5-97060-978-1.
9. Документация Django REST Framework. — URL: <https://www.django-rest-framework.org/>.
10. Sanders, D. Django REST Framework Simple JWT Documentation. — Version 5.5.1. — URL: <https://django-rest-framework-simplejwt.readthedocs.io/>.
11. Документация Django Channels. — URL: <https://channels.readthedocs.io/>.
12. Django REST Framework Documentation — Serializers. — URL: <https://www.django-rest-framework.org/api-guide/serializers/>.
13. Project Stage Academy. Notification System for New Messages (GitHub Issue #73). — 2025. — URL: https://github.com/Project-Stage-Academy/ChNU_PM_2025/issues/73.
14. Zoma00. Django Notifications: Real-Time Notifications with WebSockets. — GitHub Repository. — 2025. — URL: <https://github.com/zoma00/Django-notifications>.
15. Документация TanStack React Query. — URL: <https://tanstack.com/query/latest>.
16. Документация TanStack React Query: Optimistic Updates. — URL: <https://tanstack.com/query/v5/docs/framework/react/guides/optimistic-updates>.
17. Документация Axios. — URL: <https://axios-http.com/>.
18. Документация React Router. — URL: <https://reactrouter.com/>.
19. Документация PostgreSQL. — URL: <https://www.postgresql.org/docs/>.
20. Документация Redis. — URL: <https://redis.io/documentation>.

21. Документация PlantUML. — URL: <https://plantuml.com/>.
22. Henderson, J. P. PlantUML Examples for UML, ERD, Gantt Charts and More. — GitHub Repository. — URL: <https://github.com/joelparkerhenderson/plantuml-examples>.
23. MDN Web Docs. WebSocket API. — URL: <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>.
24. React Documentation — Hooks: useContext, useReducer. — URL: <https://react.dev/reference/react/useContext>.
25. Официальная документация Python для Django. — URL: <https://docs.djangoproject.com/>.

7. Контрольные сроки представления отдельных разделов курсовой работы:

- 25 % - предоставление первого раздела «12» марта 2026 г.
- 50 % - предоставление второго раздела «09» апреля 2026 г.
- 75 % - предоставление третьего раздела «30» апреля 2026 г.
- 100 % - предоставление работы на отзыв «14» мая 2026 г.

8. Срок защиты студентом курсовой работы «23» мая 2026 г.

Дата выдачи задания «12» февраля 2026 г.

Руководитель курсового проекта

К.т.н.
(ученая степень, звание)

(личная подпись)

Ф.В. Самойлов
(инициалы, фамилия)

Задание принял к исполнению студент очной формы обучения 2 курса группы

ПИЖ-б-о-24-1

(личная подпись)

Н.И. Усенко
(инициалы, фамилия)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	10
Актуальность	10
Цель и задачи	10
Объект и предмет исследования	11
1. АНАЛИТИЧЕСКАЯ ЧАСТЬ.....	12
1.1. Описание предметной области	12
1.2. Анализ бизнес-процессов (IDEF0).....	12
1.3. SWOT-анализ текущего состояния.....	13
1.4. Анализ аналогов и конкурентов	14
1.5. Обоснование необходимости разработки.....	15
1.6. Экономическое обоснование (ROI).....	15
2. ПРОЕКТНАЯ ЧАСТЬ	17
2.1. Модель требований к ПО	17
2.1.1. Use Case диаграмма	18
2.1.2. Матрица стейкхолдеров	18
2.1.3. Спецификация прецедентов.....	19
2.1.4. Глоссарий терминов.....	20
2.2. Модель предметной области	21
2.2.1. Domain Model (диаграмма классов)	21
2.2.2. Описание сущностей и атрибутов.....	21
2.2.3. Бизнес-правила.....	22
2.3. Архитектурное проектирование	23
2.3.1. Выбор архитектурного стиля.....	23
2.3.2. Диаграмма компонентов.....	23
2.3.3. Описание слоёв и их ответственности	24
2.3.4. Схема взаимодействия.....	25

2.4. Проектирование базы данных.....	25
2.4.1. ER-диаграмма.....	25
2.4.2. Физическая модель данных	25
2.4.3. DDL-скрипты.....	26
2.5. Детальное проектирование	26
2.5.1. Диаграммы последовательности	26
2.5.2. Диаграммы классов проектирования.....	27
2.5.3. Применение паттернов	28
3. РЕАЛИЗАЦИОННАЯ ЧАСТЬ	29
3.1. Реализация бизнес-логики	29
3.1.1. Структура проекта Django.....	29
3.1.2. Модели-сущности	30
3.1.3. Слой доступа к данным.....	30
3.1.4. Слой управления	30
3.2. Реализация API.....	31
3.2.1. REST API эндпоинты.....	31
3.2.2. JWT-аутентификация.....	32
3.2.3. Разрешения для разграничения доступа.....	33
3.2.4. Документация API	33
3.3. Реализация WebSocket	33
3.3.1. Настройка ASGI и Channel Layers.....	33
3.3.2. WebSocket Consumer	34
3.3.3. Маршрутизация WebSocket.....	34
3.4. Реализация фронтенда.....	34
3.4.1. Структура проекта React	34
3.4.2. Компоненты	35

3.4.3. Маршрутизация.....	36
3.4.4. Управление состоянием.....	36
3.4.5. Взаимодействие с API	37
3.4.6. WebSocket клиент.....	37
3.5. Расширенная клиентская логика	37
3.5.1. Оптимистичные обновления.....	37
3.5.2. Кэширование запросов	38
3.5.3. Пагинация, фильтрация, поиск на клиенте	38
3.5.4. Real-time уведомления.....	38
3.6. Рефакторинг и оптимизация	39
3.6.1. Статический анализ кода.....	39
3.6.2. Оптимизация запросов	39
3.6.3. Оптимизация React	40
3.7. Безопасность и транзакции	40
3.7.1. JWT-аутентификация с refresh токенами.....	40
3.7.2. Защита от XSS, CSRF	40
3.7.3. Валидация данных на сервере и клиенте	41
3.7.4. Аутентификация WebSocket.....	41
4. ТЕСТИРОВАНИЕ И ОБЕСПЕЧЕНИЕ КАЧЕСТВА.....	42
4.1. Модульное тестирование бэкенда	42
4.2. Модульное тестирование фронтенда	43
4.3. Интеграционное тестирование	43
4.4. Нагрузочное тестирование WebSocket.....	44
4.5. Результаты тестирования.....	44
5. РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ.....	46
5.1. Инструкция по установке.....	46

5.2. Инструкция по настройке.....	46
5.3. Требования к окружению	47
6. УПРАВЛЕНИЕ ПРОЕКТОМ.....	49
6.1. WBS (иерархическая структура работ).....	49
6.2. Диаграмма Ганта	49
6.3. Оценка трудозатрат (COCOMO).....	50
6.4. Управление рисками	50
ЗАКЛЮЧЕНИЕ	52
Выводы	52
Результаты	52
Перспективы развития.....	52
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	53
ПРИЛОЖЕНИЯ	56

ВВЕДЕНИЕ

Актуальность

В современном цифровом обществе навык быстрой и безошибочной печати на клавиатуре становится не просто профессиональным требованием для IT-специалистов, журналистов, копирайтеров и секретарей, но и важной составляющей общей цифровой грамотности. Большинство пользователей проводят за компьютером значительную часть времени, однако скорость печати многих из них остаётся низкой, что приводит к снижению продуктивности, повышенной утомляемости и ошибкам при вводе текста. Существующие онлайн-тренажёры часто либо перегружены рекламой, либо требуют платной подписки, либо не предоставляют достаточной статистики для отслеживания прогресса. В связи с этим разработка собственного веб-приложения клавиатурного онлайн-тренажёра, учитывающего потребности различных категорий пользователей и предоставляющего наглядную обратную связь, является актуальной задачей, позволяющей повысить эффективность обучения слепому десятипальцевому методу печати.

Цель и задачи

Цель: повышение эффективности обучения навыку слепого метода печати на клавиатуре за счёт применения структурированных упражнений и объективной оценки результатов, что позволяет снизить зависимость от несистематизированных методов обучения и ускорить формирование устойчивого навыка.

Задачи:

1. Провести предпроектное обследование и бизнес-анализ предметной области.
2. Разработать модель требований к программному обеспечению.
3. Спроектировать архитектуру клиент-серверного приложения.
4. Реализовать бэкенд на Django REST Framework с REST API.

5. Реализовать систему JWT-аутентификации (access/refresh токены).
6. Настроить разрешения (permissions) для разграничения доступа.
7. Реализовать фронтенд на React с использованием React Router и Axios.
8. Настроить CORS и интеграцию между фронтендом и бэкендом.
9. Обеспечить качество кода через тестирование.
10. Реализовать расширенную клиентскую логику (оптимистичные обновления).
11. Добавить кэширование запросов (React Query / RTK Query).
12. Реализовать пагинацию, фильтрацию и поиск на клиенте.
13. Внедрить WebSocket (Django Channels) для real-time уведомлений.

Объект и предмет исследования

Объект исследования: процесс обучения навыку слепой печати на клавиатуре с использованием веб-технологий, включая методы оценки скорости набора текста, точности ввода и анализа динамики улучшения результатов.

Предмет исследования: методы и алгоритмы реализации веб-приложения клавиатурного тренажёра, включая способы сравнения вводимого текста с эталонным, расчёт метрик, организацию пользовательских сессий и хранение статистических данных в реляционной базе данных.

1. АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1. Описание предметной области

Предметная область проекта связана с обучением слепой печати и самостоятельной тренировкой скорости набора текста. Пользователь выбирает язык, режим тренировки, количество слов либо конкретный урок, после чего вводит предложенный текст. Система фиксирует вводимые символы, сравнивает их с эталоном и рассчитывает показатели WPM, CPM, точность, количество ошибок и длительность сессии.

Особенностью выбранной области является необходимость немедленной обратной связи. Пользователь должен видеть активное слово, правильные и ошибочные символы, подсветку следующей клавиши, а после завершения - итоговые метрики. Для зарегистрированных пользователей важна история результатов, так как она позволяет отслеживать прогресс и личные рекорды по языкам и количеству слов.

В разработанном проекте предметная область представлена тремя базовыми сущностями: пользователь, урок и результат теста. Урок содержит язык, название, комбинацию клавиш и тренировочный текст. Результат теста принадлежит пользователю и хранит метрики прохождения. Дополнительно реализован канал уведомлений, который сообщает пользователю о новом личном рекорде в реальном времени.

1.2. Анализ бизнес-процессов (IDEF0)

Основной бизнес-процесс системы можно представить как преобразование входных действий пользователя в измеримый результат обучения. На вход поступают выбранные параметры тренировки и последовательность нажатий клавиш. Управляющими воздействиями выступают правила валидации уроков, допустимые языки, настройки доступа и критерии расчета метрик (<https://github.com/NikolayUsenko/fourth-term-course-project/blob/main/docs/contextual-business-process-model.png>). Механизмами

являются клиентское React-приложение, серверное API, WebSocket-слой и база данных.

Декомпозиция процесса включает выбор режима, получение тренировочного материала, ввод текста, расчет статистики, сохранение результата и отправку уведомления о рекорде. Для гостя процесс завершается локальным показом результата, а для авторизованного пользователя дополнительно выполняется запись результата и обновление статистики.

1.3. SWOT-анализ текущего состояния

Для системной оценки текущего состояния проекта проведём SWOT-анализ, который позволит выявить внутренние сильные и слабые стороны разрабатываемого решения, а также внешние возможности и угрозы (таблица 1).

Таблица 1 – SWOT-анализ проекта

Категория	Содержание
Сильные стороны	SPA-интерфейс без перезагрузки страниц; поддержка русского и английского языков; JWT-аутентификация; сохранение истории результатов; уведомления о личном рекорде через WebSocket.
Слабые стороны	Учебная версия использует SQLite и InMemory Channel Layer; отсутствует полноценная панель аналитики администратора во frontend; набор слов и уроков ограничен подготовленными данными.
Возможности	Расширение библиотеки уроков; переход на PostgreSQL и Redis; добавление рейтингов, адаптивных упражнений и экспорта статистики.
Угрозы	Конкуренция со стороны зрелых онлайн-тренажеров; риск потери мотивации пользователя без геймификации; необходимость защиты токенов и персональных результатов.

1.4. Анализ аналогов и конкурентов

На рынке существуют тренажеры печати общего назначения, однако они часто ориентированы либо на массовую игровую практику, либо на закрытую образовательную платформу (таблица 2). Для учебного проекта важно получить управляемую архитектуру, доступ к исходному коду и возможность развивать серверную часть, API и real-time взаимодействие.

Таблица 2 – Сравнение с аналогами

Аналог	Ключевые возможности	Ограничения относительно проекта
Ratatype	Онлайн-уроки, сертификаты, базовая статистика.	Закрытая платформа; ограниченная адаптация под собственные бизнес-правила и API.
TypingClub	Развитая система уроков, образовательные сценарии, классы.	Ориентирован на готовый сервис; интеграция и модификация логики ограничены.
Monkeytype	Гибкие тесты скорости, разные режимы, современный UI.	Фокус на тестировании скорости, а не на управляемых учебных уроках и серверной статистике учебного проекта.
Keybr	Адаптивная генерация упражнений, анализ ошибок.	Нет учебной интеграции с собственной backend-архитектурой и заданными ролями доступа.

1.5. Обоснование необходимости разработки

Необходимость разработки обусловлена учебной и практической ценностью проекта. Готовые сервисы позволяют тренироваться, но не дают возможности изучить полный цикл создания веб-приложения: проектирование предметной модели, настройку REST API, реализацию JWT-аутентификации, маршрутизацию SPA, работу с WebSocket и серверную агрегацию статистики.

Собственное приложение позволяет адаптировать набор уроков под конкретную программу обучения, хранить результаты в контролируемой базе данных и развивать функциональность без зависимости от внешнего сервиса. Кроме того, проект демонстрирует типовую клиент-серверную архитектуру, актуальную для образовательных, корпоративных и тренировочных веб-систем.

1.6. Экономическое обоснование (ROI)

Экономическая эффективность для учебного проекта оценивается ориентировочно, так как приложение не выводится на коммерческий рынок. Основной эффект заключается в снижении затрат на подключение сторонних сервисов, возможности повторного использования кода в учебном процессе и накоплении собственной статистики пользователей (таблица 3).

Таблица 3 – Ориентировочная оценка экономической эффективности

Показатель	Оценка	Комментарий
Трудозатраты разработки	160 чел.-ч.	Проектирование, backend, frontend, WebSocket, тестирование и документация.
Условная ставка	500 руб./ч.	Учебная расчетная ставка для оценки себестоимости.
Стоимость разработки	80 000 руб.	160 x 500 руб./ч.
Годовой эффект	140 000 руб.	Экономия на лицензиях, настройке сторонних решений и

		повторном использовании в группе.
ROI	75 %	$(140\,000 - 80\,000) / 80\,000 \times 100 \%$.

Полученное значение ROI показывает, что при повторном использовании приложения в образовательном процессе разработка является оправданной. Для промышленной эксплуатации расчет необходимо уточнить с учетом хостинга, сопровождения, резервного копирования, мониторинга и расходов на информационную безопасность.

2. ПРОЕКТНАЯ ЧАСТЬ

2.1. Модель требований к ПО

Требования сформированы на основе реализованного проекта и делятся на функциональные, нефункциональные и ограничения среды (таблица 4). Основной пользовательский поток включает выбор режима, ввод текста, расчет результата, сохранение статистики и получение уведомления о рекорде.

Таблица 4 – Требования к программному обеспечению

ID	Требование	Реализация в проекте
FR-1	Пользователь может проходить тест скорости печати без регистрации.	Страница TestPage и хук useTypingTest.
FR-2	Система поддерживает русский и английский языки.	Поля language в Lesson и TestResult, словари EN_WORDS/RU_WORDS.
FR-3	Пользователь может выбирать 10, 25, 50 или 100 слов.	WORD_COUNTS во frontend и validate_word_count на backend.
FR-4	Авторизованный пользователь может сохранять результаты.	POST /api/results/ через TestResultViewSet.
FR-5	Система рассчитывает WPM, CPM, точность, ошибки и длительность.	useTypingTest и useLessonTyping.
FR-6	Пользователь может просматривать уроки и проходить выбранный урок.	LessonsPage, LessonPracticePage, LessonViewSet.

FR-7	Пользователь может просматривать личную статистику.	GET /api/stats/ и StatsPage.
FR-8	Администратор может управлять уроками.	LessonViewSet с IsAdminOrReadOnly и Django Admin.
FR-9	При новом личном рекорде отправляется real-time уведомление.	Django Channels, NotificationConsumer, WebSocketService.
NFR-1	API должно защищать персональные результаты.	JWT, IsAuthenticated, фильтрация результатов по request.user.
NFR-2	Интерфейс должен работать как SPA.	React Router без полной перезагрузки страниц.
NFR-3	Данные уроков должны кэшироваться на клиенте.	TanStack React Query со staleTime.

2.1.1. Use Case диаграмма

Use Case диаграмма отражает три типа акторов: гость, авторизованный пользователь и администратор (<https://github.com/NikolayUsenko/fourth-term-course-project/blob/main/docs/use-case-diagram.png>). Гость может выполнять базовую тренировку и просматривать уроки, авторизованный пользователь получает сохранение статистики и уведомления, администратор управляет учебным контентом.

2.1.2. Матрица стейкхолдеров

Для выявления ключевых участников проекта и определения их интересов, влияния и ожиданий от разрабатываемого решения представлена матрица

2.1.3. Спецификация прецедентов

Для детализации функциональных требований к проекту разработана спецификация основных прецедентов (таблица 5).

Таблица 5 – Спецификация основных прецедентов

Прецедент	Актор	Предусловие	Основной результат
Пройти тест скорости	Гость или пользователь	Открыта главная страница.	Показаны WPM, CPM, точность, ошибки и длительность.
Сохранить результат	Пользователь	Пользователь авторизован, тест завершен.	Создана запись TestResult, определен признак личного рекорда.
Пройти урок	Гость или пользователь	Выбран существующий урок.	Пользователь вводит фиксированный тренировочный текст и получает метрики урока.
Просмотреть статистику	Пользователь	Пользователь авторизован.	Отображены агрегированные результаты по языкам и количеству слов.
Управлять уроками	Администратор	У пользователя is_staff=True.	Администратор создает, изменяет или удаляет уроки.

Получить уведомление	Пользователь	Открыто WebSocket-соединение и побит предыдущий рекорд.	На клиент отправлено сообщение типа new_record.
----------------------	--------------	---	---

2.1.4. Глоссарий терминов

Для обеспечения однозначного понимания терминов, используемых в проектной документации и при разработке системы, составлен глоссарий (таблица 6).

Таблица 6 – Глоссарий проекта

Термин	Определение
WPM	Words per minute - количество полностью правильно набранных слов в минуту.
CPM	Characters per minute - количество правильно набранных символов в минуту.
Accuracy	Точность ввода: отношение правильных нажатий к общему числу нажатий.
Lesson	Учебное упражнение с языком, комбинацией клавиш и набором слов.
TestResult	Сохраненный результат теста скорости печати авторизованного пользователя.
JWT	JSON Web Token, используемый для access/refresh-аутентификации.
Channel Layer	Промежуточный слой Django Channels для отправки событий WebSocket-потребителям.

SPA	Single Page Application - клиентское приложение, где переходы выполняются без полной перезагрузки страницы.
-----	---

2.2. Модель предметной области

Модель предметной области выделяет устойчивые понятия, необходимые для реализации тренажера. Пользователь связан с множеством результатов тестов; уроки используются как источник фиксированного тренировочного материала; сессия печати является временным клиентским объектом, который после завершения может породить сохраняемый результат.

2.2.1. Domain Model (диаграмма классов)

В рамках проектирования архитектуры программного обеспечения была построена диаграмма классов доменной модели (<https://github.com/NikolayUsenko/fourth-term-course-project/blob/main/docs/domain-model.png>).

2.2.2. Описание сущностей и атрибутов

Для формализации модели данных разрабатываемой системы и обеспечения единого представления о структуре хранения информации составлено описание основных сущностей предметной области (таблица 7).

Таблица 7 – Описание сущностей предметной области

Сущность	Назначение	Основные атрибуты
User	Учетная запись пользователя.	username, email, first_name, last_name, is_staff, is_active.
Lesson	Тренировочный урок для конкретного языка и комбинации клавиш.	language, title, key_combination, content, order, created_at, updated_at.

TestResult	Результат прохождения теста скорости печати.	user, language, word_count, wpm, cpm, accuracy, typos, duration, is_record, created_at.
TypingSession	Клиентское состояние текущей тренировки.	words, currentWordIdx, currentInput, typedHistory, status, stats.
Notification	Событие, отправляемое пользователю через WebSocket.	type, wpm, language, word_count, previous_best.

2.2.3. Бизнес-правила

Для формализации ограничений и логики работы приложения, не зависящей от конкретной технической реализации, сформулированы бизнес-правила (таблица 8).

Таблица 8 – Бизнес-правила приложения

ID	Правило
BR-1	Для английского урока допускаются только символы a-z без пунктуации; для русского урока - символы а-я и е/ё.
BR-2	Количество слов в тесте может принимать только значения 10, 25, 50 и 100.
BR-3	Результат теста сохраняется только для авторизованного пользователя.
BR-4	Пользователь видит только собственные результаты; администратор может просматривать все результаты.
BR-5	Первый результат в группе language + word_count считается личным рекордом, но уведомление отправляется только при улучшении предыдущего рекорда.

BR-6	Backspace в режиме набора намеренно не обрабатывается: тренировка является однонаправленной и оценивает точность текущего ввода.
BR-7	Уроки доступны на чтение всем пользователям, а создание и изменение разрешены только администраторам.

2.3. Архитектурное проектирование

Приложение спроектировано как клиент-серверная система. Клиент отвечает за интерактивный ввод, визуализацию клавиатуры и отображение статистики. Сервер отвечает за учетные записи, хранение уроков, сохранение результатов, агрегацию статистики и отправку WebSocket-событий.

2.3.1. Выбор архитектурного стиля

Клиент-серверный стиль выбран из-за четкого разделения интерфейса и бизнес-логики. React SPA обеспечивает быстрый пользовательский интерфейс, Django REST Framework предоставляет единый HTTP API, а Django Channels добавляет асинхронный канал уведомлений. REST используется для команд и запросов, WebSocket – только для событий, которые должны появляться у пользователя без ручного обновления страницы.

Такое разделение снижает связанность компонентов: фронтенд может развиваться независимо от серверных моделей, сервер контролирует безопасность и валидацию, а WebSocket-слой подключается только к сценариям, где требуется real-time поведение.

2.3.2. Диаграмма компонентов

Для документирования физической архитектуры разрабатываемой системы была построена диаграмма компонентов (<https://github.com/NikolayUsenko/fourth-term-course-project/blob/main/docs/component-diagram.png>).

2.3.3. Описание слоёв и их ответственности

Для обеспечения модульности, удобства сопровождения и масштабирования проекта выполнено архитектурное разделение системы на слои в соответствии с принципами многоуровневой архитектуры (таблица 9).

Таблица 9 – Ответственность архитектурных слоев

Слой	Компоненты	Ответственность
Представление	React pages, components, CSS	Отображение интерфейса, обработка ввода, визуальная обратная связь.
Клиентская логика	useTypingTest, useLessonTyping, AuthContext, React Query	Состояние тренировки, авторизация, кэширование и получение данных.
API-клиент	Axios API service	Передача JWT, refresh access-токена, обработка ошибок HTTP.
REST API	DRF ViewSet, function-based views	CRUD уроков, создание результатов, выдача статистики.
Валидация	Serializers, permissions	Проверка входных данных и прав доступа.
Real-time	WebSocketService, NotificationConsumer, Channel Layer	Доставка события о новом рекорде пользователю.
Данные	Django ORM, SQLite/PostgreSQL	Хранение пользователей, уроков и результатов.

2.3.4. Схема взаимодействия

JWT-токены используются для защищенных HTTP-запросов и подключения WebSocket. Access-токен добавляется Axios-interceptor в заголовок Authorization. При ответе 401 клиент пытается обновить access-токен через refresh-токен. Для WebSocket access-токен передается в query string, после чего NotificationConsumer проверяет его через Simple JWT и присоединяет соединение к группе конкретного пользователя.

После завершения теста клиент отправляет POST /api/results/. Сервер сравнивает новый WPM с предыдущим максимумом пользователя для выбранного языка и количества слов. Если рекорд улучшен, событие отправляется в channel group, а затем доставляется активному WebSocket-клиенту.

2.4. Проектирование базы данных

База данных отражает минимальную предметную модель тренажера. Пользователь хранится в расширенной модели users.User, уроки - в core.Lesson, результаты тестов - в core.TestResult. В учебной конфигурации используется SQLite, а в .env.example предусмотрены параметры для PostgreSQL.

2.4.1. ER-диаграмма

Для формализации структуры хранения данных была разработана ER-диаграмма (<https://github.com/NikolayUsenko/fourth-term-course-project/blob/main/docs/ER-diagram-database.png>).

2.4.2. Физическая модель данных

Для реализации хранения данных в целевой СУБД разработана физическая модель данных (таблица 10).

Таблица 10 – Физическая модель данных

Таблица	Ключевые поля	Индексы и ограничения
---------	---------------	-----------------------

users_user	id, username, email, first_name, last_name, is_staff	email уникален; стандартные ограничения Django AbstractUser.
core_lesson	id, language, title, key_combination, content, order	Индексы по language и order; сортировка по language, order.
core_testresult	id, user_id, language, word_count, wpm, cpm, accuracy, typos, duration, is_record	FK на users_user; индекс user + language + word_count; индексы по language, word_count, created_at.

2.4.3. DDL-скрипты

Фактические DDL-скрипты генерируются Django migrations. Упрощенное представление ключевых таблиц, соответствующее моделям проекта и связям между ними (<https://github.com/NikolayUsenko/fourth-term-course-project/blob/main/docs/DDL-scripts.png>).

2.5. Детальное проектирование

Детальное проектирование описывает ключевые последовательности выполнения и соответствие программных классов выбранным архитектурным решениям. Основные сценарии связаны с аутентификацией, прохождением теста, сохранением результата и доставкой уведомления.

2.5.1. Диаграммы последовательности

Сценарий аутентификации начинается на странице входа, проходит через AuthContext и Axios API service, а завершается получением access/refresh-токенов и подключением WebSocket-сервиса (<https://github.com/NikolayUsenko/fourth-term-course-project/blob/main/docs/JWT-authentication-sequence.png>).

Сценарий сохранения результата объединяет CRUD-операцию создания TestResult и WebSocket-уведомление (<https://github.com/NikolayUsenko/fourth-term-course-project/blob/main/docs/sequence-saving-result-and-notification.png>).

Серверная логика сначала проверяет личный максимум, затем сохраняет результат и при необходимости отправляет событие new_record.

2.5.2. Диаграммы классов проектирования

Для документирования ключевых элементов программной реализации в терминах объектно-ориентированного проектирования представлены основные классы и модули, используемые на backend и frontend (таблица 11).

Таблица 11 – Классы и модули проектирования

Класс/модуль	Тип	Назначение
Lesson, TestResult	Django Model	Описывают основные данные предметной области и ограничения хранения.
LessonSerializer, TestResultCreateSerializer	DRF Serializer	Преобразуют модели в JSON и валидируют входные данные.
LessonViewSet, TestResultViewSet	DRF ViewSet	Реализуют CRUD и правила доступа к данным.
NotificationConsumer	AsyncWebsocketConsumer	Аутентифицирует WebSocket и отправляет сообщения пользователю.
AuthContext	React Context	Хранит состояние пользователя, выполняет login/logout/register.
useTypingTest, useLessonTyping	React Hook	Инкапсулируют состояние и алгоритм набора текста.
api.js	Axios service	Добавляет JWT к запросам и обновляет access-токен при 401.

websocket.js	Service object	Управляет WebSocket-подключением и подписчиками уведомлений.
--------------	----------------	--

2.5.3. Применение паттернов

Для обеспечения гибкости, повторного использования кода и снижения связанности компонентов в проекте использован ряд проектных паттернов (таблица 12).

Таблица 12 – Примененные проектные паттерны

Паттерн	Где применен	Эффект
ViewSet	LessonViewSet, TestResultViewSet	Единая организация CRUD-операций и маршрутов DRF.
Serializer	LessonSerializer, TestResultSerializer	Разделение модели хранения и формата обмена по API.
Repository через ORM	Django QuerySet и aggregate	Доступ к данным без ручного SQL в бизнес-логике.
React Hooks	useTypingTest, useLessonTyping, useWebSocket	Повторное использование клиентской логики и изоляция состояния.
Observer	WebSocketService.subscribe	Компонент Notification подписывается на события WebSocket.
Interceptor	Axios request/response interceptors	Централизованное добавление токена и refresh при 401.

3. РЕАЛИЗАЦИОННАЯ ЧАСТЬ

3.1. Реализация бизнес-логики

Backend реализован как Django-проект config с приложениями core и users. Приложение users отвечает за расширенную модель пользователя, регистрацию и профиль. Приложение core содержит уроки, результаты тестов, статистику, права доступа и WebSocket consumer.

3.1.1. Структура проекта Django

Для обеспечения навигации по серверной части приложения и понимания назначения ключевых файлов представлена структура backend-проекта на Django (таблица 13).

Таблица 13 – Структура backend-проекта

Файл/пакет	Назначение
config/settings.py	Настройки Django, DRF, JWT, CORS, Channels и базы данных.
config/urls.py	Подключение admin, /api/ и /api/auth/.
config/asgi.py	ProtocolTypeRouter для HTTP и WebSocket.
config/routing.py	Маршрут /ws/notifications/.
users/models.py	Расширенная модель User с уникальным email.
users/views.py	Регистрация и профиль пользователя.
core/models.py	Модели Lesson и TestResult.
core/views.py	ViewSet уроков и результатов, агрегированная статистика.
core/consumers.py	WebSocket consumer уведомлений.
core/tests.py	Набор backend-тестов для уроков, результатов и статистики.

3.1.2. Модели-сущности

Модель Lesson хранит язык, название, комбинацию клавиш, тренировочный текст и порядок вывода. Для языка используются значения en и ru. Валидация содержимого урока вынесена в serializer, что позволяет нормализовать регистр и исключить недопустимые символы.

Модель TestResult хранит метрики заверченного теста: WPM, CPM, точность, количество ошибок, длительность и признак личного рекорда. Связь с пользователем реализована через ForeignKey на AUTH_USER_MODEL с related_name test_results.

3.1.3. Слой доступа к данным

Доступ к данным реализован через Django ORM. Для получения статистики применяются агрегатные функции Avg, Sum, Max и Count. В TestResultViewSet администратор получает все результаты с select_related('user'), а обычный пользователь - только записи, отфильтрованные по request.user.

При сохранении результата сервер вычисляет previous_best через aggregate(Max('wpm')) по группе user + language + word_count. Это позволяет определить, является ли новая запись личным рекордом, без загрузки всей истории пользователя в память приложения.

3.1.4. Слой управления

LessonViewSet предоставляет CRUD для уроков. Чтение доступно всем пользователям, а изменение разрешено только администраторам через IsAdminOrReadOnly. TestResultViewSet разрешает только GET и POST, что делает результаты неизменяемыми после создания.

Serializer-слой разделяет чтение и запись результатов: TestResultSerializer возвращает username и read-only поля, а TestResultCreateSerializer принимает только метрики, устанавливая user и is_record на сервере. Такой подход исключает подмену владельца результата со стороны клиента.

3.2. Реализация API

REST API построено на Django REST Framework. Роутер автоматически формирует стандартные маршруты для уроков и результатов, а отдельные function-based views отвечают за агрегированную статистику пользователя и административную статистику.

3.2.1. REST API эндпоинты

Для документирования всех доступных точек интеграции frontend-приложения с серверной частью представлен перечень REST API эндпоинтов, реализованных средствами Django REST Framework, с указанием метода, маршрута, назначения и уровня доступа (таблица 14).

Таблица 14 – REST API эндпоинты

Метод	Маршрут	Назначение	Доступ
GET	/api/lessons/	Список уроков, фильтрация language, ordering.	Публичный
POST	/api/lessons/	Создание урока.	Администратор
GET	/api/lessons/{id}/	Получение урока.	Публичный
PUT/PATCH/DELETE	/api/lessons/{id}/	Изменение или удаление урока.	Администратор
GET	/api/results/	Список результатов пользователя.	Авторизованный
POST	/api/results/	Сохранение результата теста.	Авторизованный

GET	/api/stats/	Агрегированная личная статистика.	Авторизованный
GET	/api/admin/stats/	Статистика пользователей.	Администратор
POST	/api/auth/register/	Регистрация.	Публичный
POST	/api/auth/login/	Получение access/refresh токенов.	Публичный
POST	/api/auth/token/refresh/	Обновление access-токена.	Публичный с refresh
POST	/api/auth/logout/	Blacklist refresh-токена.	Авторизованный
GET/PATCH	/api/auth/profile/	Просмотр и изменение профиля.	Авторизованный

3.2.2. JWT-аутентификация

JWT-аутентификация реализована через `django-rest-framework-simplejwt`. Access-токен живет ограниченное время и передается в заголовке `Authorization: Bearer`. Refresh-токен хранится на клиенте и используется для получения нового access-токена при ответе 401.

Во frontend логика refresh централизована в `response interceptor` файла `services/api.js`. Если одновременно несколько запросов получают 401, они помещаются в `pendingQueue` и повторяются после успешного обновления токена. Это предотвращает лишние параллельные запросы refresh.

3.2.3. Разрешения для разграничения доступа

Для уроков используется пользовательское разрешение `IsAdminOrReadOnly`: безопасные методы доступны всем, а операции изменения - только `staff`-пользователям. Для результатов применяется `IsAuthenticated`, а `queryset` дополнительно ограничивает данные текущим пользователем.

Административная статистика защищена `IsAdminUser`. Профиль пользователя возвращает объект `request.user`, что исключает доступ к чужому профилю через подмену идентификатора в URL.

3.2.4. Документация API

В проекте реализована автоматическая генерация интерактивной документации API с использованием библиотеки `drf-spectacular`. `drf-spectacular` полностью решает задачу автоматической генерации документации API. Клиентская интеграция может выполняться на основе стабильной `OpenAPI`-схемы, а разработчики получают удобный инструмент для отладки и демонстрации функционала.

3.3. Реализация WebSocket

`WebSocket` используется для доставки уведомлений о новых личных рекордах. Канал является `read-only` со стороны клиента: пользователь не отправляет сообщения, а только получает события от сервера.

3.3.1. Настройка ASGI и Channel Layers

`ASGI_APPLICATION` указывает на `config.asgi.application`. В `ASGI`-конфигурации `ProtocolTypeRouter` разделяет HTTP и `WebSocket`. `WebSocket`-запросы проходят через `AuthMiddlewareStack` и `URLRouter` с маршрутом `/ws/notifications/`.

В учебной конфигурации `CHANNEL_LAYERS` использует `InMemoryChannelLayer`. Для промышленной эксплуатации рекомендуется

RedisChannelLayer, так как InMemory-слой не подходит для нескольких процессов и серверов.

3.3.2. WebSocket Consumer

NotificationConsumer извлекает access-токен из query string, проверяет его через AccessToken Simple JWT и получает активного пользователя. После успешной проверки соединение добавляется в группу user_{id}_notifications.

Метод new_record принимает событие channel layer и отправляет клиенту JSON-объект вида {type: 'new_record', data: {...}}. Если токен отсутствует или недействителен, consumer закрывает соединение кодом 4001.

3.3.3. Маршрутизация WebSocket

Маршрут WebSocket определен в config/routing.py: path('ws/notifications/', NotificationConsumer.as_asgi()). Клиент подключается к ws://localhost:8000/ws/notifications/?token=<access_token>, адрес сервера задается переменной REACT_APP_WS_URL.

3.4. Реализация фронтенда

Frontend реализован как React SPA. Пользовательские маршруты определены в App.jsx, состояние авторизации - в AuthContext, запросы к API - в services/api.js, а интерактивная логика печати вынесена в отдельные хуки.

3.4.1. Структура проекта React

Для обеспечения навигации по клиентской части приложения и понимания организации кода представлена структура frontend-проекта на React (таблица 15).
Таблица 15 – Структура frontend-проекта

Папка/файл	Назначение
------------	------------

src/pages	Страницы TestPage, LessonsPage, LessonPracticePage, StatsPage, LoginPage, RegisterPage.
src/components	Navbar, Keyboard, ProtectedRoute, Notification, модальные окна результатов.
src/hooks	useTypingTest, useLessonTyping, useWebSocket.
src/contexts/AuthContext.js	Глобальное состояние авторизации и подключение WebSocket.
src/services/api.js	Axios instance, JWT interceptor, refresh очередь.
src/services/websocket.js	Сервис WebSocket с подписчиками и переподключением.
src/utils/wordGenerator.js	Генерация случайных слов для русского и английского тестов.
src/App.css, src/index.css	Стили интерфейса, страниц, клавиатуры и таблиц статистики.

3.4.2. Компоненты

Для систематизации клиентской части приложения и понимания роли каждого элемента интерфейса представлен перечень основных React-компонентов (таблица 16).

Таблица 16 – Основные React-компоненты

Компонент	Назначение
Navbar	Основная навигация и действия входа/выхода.
Keyboard	Виртуальная клавиатура QWERTY/ЙЦУКЕН с подсветкой следующего символа.
ProtectedRoute	Защита страницы статистики от неавторизованного доступа.
Notification	Отображение WebSocket-уведомлений.
TestResultModal	Итоги теста скорости.

LessonResultModal	Итоги урока и переход к следующему уроку.
-------------------	---

3.4.3. Маршрутизация

Для организации навигации между страницами клиентского приложения представлены маршруты frontend-приложения (таблица 17).

Таблица 17 – Маршрутизация frontend-приложения

Маршрут	Компонент	Назначение
/	TestPage	Главный тест скорости печати.
/lessons	LessonsPage	Список уроков с фильтром языка.
/lessons/:id	LessonPracticePage	Прохождение конкретного урока.
/stats	ProtectedRoute + StatsPage	Личная статистика авторизованного пользователя.
/login	LoginPage	Вход.
/register	RegisterPage	Регистрация.
*	Navigate to /	Перенаправление неизвестных маршрутов.

3.4.4. Управление состоянием

Глобальное состояние авторизации хранится в AuthContext. При загрузке приложения контекст проверяет наличие access-токена, получает профиль пользователя и подключает WebSocket. Методы login, logout и register инкапсулируют взаимодействие с API.

Серверные данные уроков и статистики загружаются через TanStack React Query. Для уроков используется staleTime 10 минут, для статистики - 30 секунд. Это уменьшает количество повторных запросов и делает интерфейс отзывчивее.

Локальное состояние тренировки находится внутри страниц и хуков: язык, количество слов, активное слово, введенный текст, история введенных слов, статус и рассчитанная статистика.

3.4.5. Взаимодействие с API

`services/api.js` создает единый `Axios` instance с базовым URL из `REACT_APP_API_URL`. `Request interceptor` добавляет access-токен из `localStorage` в заголовок `Authorization`. `Response interceptor` обрабатывает 401, обновляет access через `/auth/token/refresh/` и повторяет исходный запрос.

Для предотвращения гонки refresh-запросов используется флаг `isRefreshing` и очередь `pendingQueue`. Пока один запрос обновляет токен, остальные ожидают новый access и повторяются после `processQueue`.

3.4.6. WebSocket клиент

WebSocket-клиент реализован в `services/websocket.js` как объект-сервис. Он хранит текущее соединение, набор подписчиков и таймер переподключения. При закрытии соединения сервис через 5 секунд пытается подключиться снова, если в `localStorage` есть access-токен.

Компоненты подписываются на сообщения через `subscribe(callback)`, а сервис вызывает всех подписчиков при получении валидного JSON-сообщения. Такой подход отделяет транспортный слой от визуального компонента `Notification`.

3.5. Расширенная клиентская логика

Расширенная клиентская логика сосредоточена в интерактивном вводе, кэшировании, фильтрации уроков, автоматическом refresh-токенов и real-time уведомлениях. Благодаря хукам пользователь получает мгновенную обратную связь без ожидания ответа сервера на каждое нажатие клавиши.

3.5.1. Оптимистичные обновления

В контексте тренажера оптимистичное поведение реализовано на уровне клиентской сессии: подсветка символов, активное слово, прокрутка строки, виртуальная клавиатура и итоговые метрики обновляются сразу в браузере.

Сервер участвует только после завершения теста, когда нужно сохранить итоговый результат.

Для персистентных данных выбран более надежный вариант: результат не добавляется в историю до подтверждения POST /api/results/. Это предотвращает отображение несуществующих записей при ошибке сети или истекшей авторизации.

3.5.2. Кэширование запросов

React Query кэширует списки уроков по ключу ['lessons', language] и личную статистику по ключу ['user-stats']. Это позволяет переключаться между страницами без лишней загрузки данных, пока кэш считается свежим.

Для страницы прохождения урока используется отдельный ключ ['lesson', id], что позволяет повторно открывать урок без немедленного повторного запроса. После завершения урока дополнительно загружается список уроков текущего языка, чтобы определить следующий урок.

3.5.3. Пагинация, фильтрация, поиск на клиенте

Backend использует PageNumberPagination с размером страницы 20. Клиент учитывает возможный формат пагинированного ответа через выражение `r.data.results ?? r.data`, поэтому корректно работает и с пагинированным, и с непагинированным списком.

Фильтрация уроков реализована по языку через query-параметр `/api/lessons/?language=en` или `/api/lessons/?language=ru`. На странице статистики пользователь переключает вкладки English/Russian, а таблица показывает агрегаты по количеству слов.

3.5.4. Real-time уведомления

Real-time уведомление отправляется только при улучшении предыдущего личного рекорда. Такой критерий снижает шум уведомлений: первый результат

сохраняется как рекорд, но сообщение отправляется только тогда, когда пользователь действительно превзошел прошлый максимум.

Сообщение содержит WPM, язык, количество слов и `previous_best`. На клиенте Notification получает событие через `WebSocketService` и отображает его независимо от текущей страницы SPA.

3.6. Рефакторинг и оптимизация

Код проекта разделен по ответственности: API-клиент и `WebSocket` вынесены в `services`, логика печати - в `hooks`, контекст авторизации - в `contexts`, страницы - в `pages`. Это упрощает сопровождение и снижает дублирование между режимом теста и режимом урока.

3.6.1. Статический анализ кода

Frontend основан на `react-scripts`, поэтому использует стандартную ESLint-конфигурацию Create React App. Для backend-части в дальнейшем рекомендуется подключить `flake8` или `ruff` и зафиксировать правила в отдельном конфигурационном файле.

В текущей структуре кода соблюдается единая организация импортов, именование компонентов и разделение логики. Для промышленной сборки также рекомендуется добавить форматирование Prettier для JS/JSX и black для Python-кода.

3.6.2. Оптимизация запросов

В `TestResultViewSet` для административного просмотра применяется `select_related('user')`, чтобы избежать дополнительных запросов при обращении к данным пользователя. Агрегированная статистика рассчитывается на уровне базы данных через `Count`, `Sum` и `Avg`.

`prefetch_related` в текущей модели не требуется, так как у Lesson нет связанных коллекций, а основной пользовательский список результатов

фильтруется по одному пользователю. При добавлении комментариев, достижений или тегов уроков `prefetch_related` станет актуальным.

3.6.3. Оптимизация React

В React-коде применяются `useCallback` и `useMemo` для обработчиков, вычисления следующего символа, определения следующего урока и предотвращения лишних пересозданий функций. Для высокочастотного ввода используются `refs`, чтобы обработчики клавиатуры не зависели от устаревших замыканий.

`React.memo` может быть добавлен для `Keyboard` и модальных окон при дальнейшем росте интерфейса. В текущей версии основная оптимизация достигается локализацией состояния и отсутствием серверных запросов на каждое нажатие клавиши.

3.7. Безопасность и транзакции

Безопасность проекта строится вокруг JWT-аутентификации, серверной валидации, ограничений прав доступа и CORS-настроек. Сервер не доверяет клиенту в вопросах владельца результата и признака личного рекорда.

3.7.1. JWT-аутентификация с refresh токенами

`Access`-токен используется для текущих запросов, `refresh`-токен - для продления сессии. В настройках `SIMPLE_JWT` включены `ROTATE_REFRESH_TOKENS` и `BLACKLIST_AFTER_ROTATION`, что снижает риск повторного использования старых `refresh`-токенов после обновления.

3.7.2. Защита от XSS, CSRF

`CORS_ALLOWED_ORIGINS` ограничивает источники frontend-приложения адресами `localhost` и `127.0.0.1` в учебной конфигурации. Так как API использует `Bearer`-токены, защита `CSRF` для API-запросов не является основным

механизмом, однако стандартный `CsrfViewMiddleware` остается включенным в `Django middleware`.

Токены хранятся в `localStorage`, что удобно для SPA, но требует аккуратной защиты от XSS. Для промышленного режима следует дополнительно рассмотреть `HttpOnly cookies`, `CSP-заголовки` и строгую проверку пользовательского HTML, если в системе появится ввод произвольного форматированного контента.

3.7.3. Валидация данных на сервере и клиенте

Серверная валидация уроков проверяет соответствие языка и символов: английский контент должен содержать только латинские буквы a-z, русский - русские буквы и е/ё. Контент нормализуется к нижнему регистру и одиночным пробелам.

Для результатов проверяется допустимое количество слов и диапазон точности от 0 до 100. Клиент также ограничивает выбор значений через фиксированные массивы `WORD_COUNTS` и `LANGUAGES`, но окончательное решение всегда принимает сервер.

3.7.4. Аутентификация WebSocket

`WebSocket`-запрос проходит через `AuthMiddlewareStack`, а затем `NotificationConsumer` дополнительно извлекает `access-токен` из `query string` и проверяет его через `Simple JWT`. Это важно, поскольку стандартная `session-аутентификация` недостаточна для SPA, работающей с `JWT`.

После проверки `token payload consumer` получает пользователя из базы и создает `group_name` на основе его `id`. Благодаря этому уведомление отправляется строго конкретному пользователю, а не всем подключенным клиентам.

4. ТЕСТИРОВАНИЕ И ОБЕСПЕЧЕНИЕ КАЧЕСТВА

4.1. Модульное тестирование бэкенда

Для обеспечения качества серверной части приложения и проверки соблюдения бизнес-правил представлен перечень backend-тестов (таблица 18).

Таблица 18 – Backend-тесты проекта

Группа тестов	Проверяемое поведение
LessonValidationTests.test_list_public	Публичный доступ к списку уроков.
test_filter_by_language	Фильтрация уроков по language=ru/en.
test_en_lesson_valid	Создание корректного английского урока.
test_en_key_combination_digits_rejected	Отклонение цифр в английской комбинации.
test_en_content_punctuation_rejected	Отклонение пунктуации в английском контенте.
test_ru_key_combination_latin_rejected	Отклонение латиницы в русском уроке.
test_regular_user_cannot_create	Запрет создания урока обычным пользователем.
TestResultTests.test_create_ok	Создание результата авторизованным пользователем.
test_unauthenticated_rejected	Запрет сохранения результата без авторизации.
test_first_result_is_record	Первый результат помечается как рекорд.
test_better_wpm_is_record	Лучший WPM становится рекордом.

test_lower_wpm_not_record	Худший WPM не становится рекордом.
test_invalid_word_count_rejected	Отклонение недопустимого word_count.
test_stats_endpoint	Расчет агрегированной статистики пользователя.

4.2. Модульное тестирование фронтенда

Для обеспечения надёжности клиентской части приложения и проверки корректной работы пользовательских сценариев представлен план frontend-тестов (таблица 19).

Таблица 19 – План frontend-тестов

Компонент/хук	Контрольный сценарий
useTypingTest	Начальный статус — idle.
useLessonTyping	Фиксированный текст урока корректно разбивается на слова и рассчитывает ассигурацию.
Keyboard	Подсвечивается клавиша, соответствующая следующему ожидаемому символу.
Notification	Сообщение new_record отображается после получения WebSocket-события.

4.3. Интеграционное тестирование

Интеграционное тестирование проверяет связку React-клиента, REST API, JWT и WebSocket. Основные сценарии: регистрация, вход, refresh access-токена, загрузка уроков, прохождение урока, сохранение результата, получение статистики и получение уведомления о рекорде (таблица 20).

Таблица 20 – Интеграционные сценарии

Сценарий	Ожидаемый результат
----------	---------------------

Регистрация и вход	Создается пользователь, возвращается профиль, токены сохраняются на клиенте.
Загрузка уроков	GET /api/lessons/?language=... возвращает отсортированный список.
Сохранение результата	POST /api/results/ создает запись, недопустимый word_count отклоняется.
Статистика	GET /api/stats/ возвращает total_tests, avg_wpm, avg_cpm, avg_accuracy и разбивку по языкам.
Refresh токена	При 401 клиент получает новый access и повторяет исходный запрос.
WebSocket-рекорд	При улучшении WPM пользователь получает сообщение new_record.

4.4. Нагрузочное тестирование WebSocket

В учебной версии применяется InMemoryChannelLayer, поэтому нагрузочное тестирование имеет ограниченную практическую ценность: такой слой рассчитан на один процесс и не отражает промышленную схему. Для нагрузочного стенда необходимо заменить его на RedisChannelLayer.

Целевые метрики нагрузочного теста: количество одновременных WebSocket-соединений, время доставки события new_record, процент ошибок подключения и поведение клиента при разрыве соединения. Клиент уже содержит механизм переподключения через 5 секунд после закрытия WebSocket.

4.5. Результаты тестирования

Для обобщения результатов проверки качества разработанного решения представлены итоги тестирования и QA по основным направлениям (таблица 21).
Таблица 21 – Итоги тестирования и QA

Направление	Результат
-------------	-----------

Backend-валидация	Покрыты правила языка, контента уроков, прав доступа и допустимого word_count.
Backend-статистика	Проверены агрегаты total_tests и разбиение по языкам и количеству слов.
Аутентификация	Реализованы login, refresh, logout и защищенный профиль.
Frontend-сценарии	Подготовлены контрольные сценарии для Jest/React Testing Library; интерактивная логика вынесена в тестируемые hooks.
WebSocket	Реализована доставка new_record; для промышленного нагрузочного теста требуется Redis.
Ограничения проверки	Для запуска автоматических тестов требуется установка зависимостей из backend/requirements.txt и frontend/package.json.

5. РАЗВЁРТЫВАНИЕ И ЭКСПЛУАТАЦИЯ

5.1. Инструкция по установке

Для развёртывания разработанного программного обеспечения на локальной машине необходимо выполнить последовательность действий:

1. `cd backend`
2. `python -m venv .env`
3. `source .env/bin/activate`
4. `pip install -r requirements.txt`
5. `cp .env.example .env`
6. `python manage.py migrate`
7. `python manage.py loaddata core/fixtures/initial_lessons.json`
8. `python manage.py runserver`
9. `cd frontend`
10. `npm install`
11. `cp .env.example .env`
12. `npm start`

После запуска backend доступен по адресу `http://localhost:8000`, frontend - по адресу `http://localhost:3000`. Для WebSocket используется адрес `ws://localhost:8000/ws/notifications/`.

5.2. Инструкция по настройке

Для корректной настройки и развёртывания проекта на различных средах представлен перечень переменных окружения (таблица 22).

Таблица 22 – Переменные окружения проекта

Переменная	Назначение
SECRET_KEY	Секретный ключ Django; в production должен быть уникальным и закрытым.

DEBUG	Режим отладки. В production должен быть False.
ALLOWED_HOSTS	Список разрешенных host-имен backend.
JWT_ACCESS_TOKEN_LIFETIME	Время жизни access-токена в минутах.
JWT_REFRESH_TOKEN_LIFETIME	Время жизни refresh-токена в минутах.
CORS_ALLOWED_ORIGINS	Разрешенные origin frontend-приложения.
DB_NAME, DB_USER, DB_PASSWORD, DB_HOST, DB_PORT	Параметры PostgreSQL для production-конфигурации.
REACT_APP_API_URL	Базовый URL REST API для frontend.
REACT_APP_WS_URL	Базовый URL WebSocket-сервера для frontend.

Для production-режима необходимо заменить InMemoryChannelLayer на RedisChannelLayer и указать Redis в CHANNEL_LAYERS. Также следует настроить CORS только на доверенные домены, отключить DEBUG и использовать HTTPS.

5.3. Требования к окружению

Для успешного развёртывания и функционирования проекта на целевой платформе представлен перечень требований к окружению (таблица 23).

Таблица 23 – Требования к окружению

Компонент	Требование
Python	Совместимая версия для Django 5.2.x и зависимостей requirements.txt.

Backend dependencies	Django, DRF, Simple JWT, Channels, Daphne, django-cors-headers, psycopg2-binary и др.
Node.js/npm	Среда для React-приложения и react-scripts.
Database	SQLite для разработки; PostgreSQL рекомендуется для production.
Channel Layer	InMemory для учебного режима; Redis для production и масштабирования.
Browser	Современный браузер с поддержкой WebSocket и ES-модулей.

6. УПРАВЛЕНИЕ ПРОЕКТОМ

6.1. WBS (иерархическая структура работ)

Для структурирования всех работ по созданию проекта и определения последовательности их выполнения представлена иерархическая структура работ (таблица 24).

Таблица 24 – WBS проекта

Этап	Состав работ
1. Анализ	Предметная область, аналоги, требования, ограничения.
2. Проектирование	Use Case, доменная модель, архитектура, ER, последовательности.
3. Backend	Модели, serializers, ViewSet, статистика, JWT, permissions, tests.
4. WebSocket	ASGI, routing, consumer, channel layer, client service.
5. Frontend	Страницы, компоненты, хуки печати, AuthContext, React Query, Axios.
6. Тестирование	Backend-тесты, интеграционные сценарии, проверка безопасности.
7. Документация	Пояснительная записка, диаграммы PlantUML, инструкция запуска.

6.2. Диаграмма Ганта

Для планирования и контроля сроков выполнения работ по проекту представлен календарный план в виде диаграммы Ганта с разбивкой по неделям (таблица 25).

Таблица 25 – Календарный план работ

Период	Работы	Результат
--------	--------	-----------

Неделя 1	Анализ предметной области и аналогов.	Сформированы требования.
Неделя 2	Проектирование моделей, API и архитектуры.	Подготовлены UML-диаграммы и структура БД.
Неделя 3	Реализация backend: модели, serializers, ViewSet, JWT.	Рабочий REST API.
Неделя 4	Реализация frontend: страницы, хуки, компоненты.	Рабочий SPA-интерфейс.
Неделя 5	Интеграция WebSocket, статистика, тесты.	Уведомления, история и QA-сценарии.
Неделя 6	Документация, рефакторинг, подготовка к защите.	Заполненная пояснительная записка и итоговый проект.

6.3. Оценка трудозатрат (COCOMO)

Для ориентировочной оценки применена базовая модель COCOMO для organic-проектов: $\text{Effort} = 2.4 \times \text{KLOC}^{1.05}$, $\text{Schedule} = 2.5 \times \text{Effort}^{0.38}$. При объеме около 2,7 KLOC расчетная трудоемкость составляет примерно 6,8 человеко-месяца, а расчетный срок - около 5,2 месяца.

Так как проект выполнен с использованием высокоуровневых фреймворков Django и React, фактические учебные трудозатраты ниже классической COCOMO-оценки. Для практического планирования проекта использована оценка 160 человеко-часов, распределенная между анализом, проектированием, реализацией, тестированием и документацией.

6.4. Управление рисками

Для идентификации, оценки и минимизации потенциальных проблем в ходе разработки и эксплуатации проекта представлен реестр рисков (таблица 26).

Таблица 26 – Реестр рисков

Риск	Вероятность	Влияние	Меры снижения
Ошибки расчета WPM/accuracy	Средняя	Высокое	Вынос логики в hooks, модульные тесты, ручные контрольные сценарии.
Истечение JWT во время работы	Высокая	Среднее	Axios refresh interceptor и очередь pendingQueue.
Потеря WebSocket- соединения	Средняя	Среднее	Автоматическое переподключение клиента, Redis в production.
Некорректный учебный контент	Средняя	Среднее	Serializer-валидация языка, символов и пробелов.
Рост нагрузки на статистику	Низкая	Среднее	Агрегация на уровне БД, индексы, PostgreSQL.
XSS при хранении токенов	Средняя	Высокое	CSP, отказ от небезопасного HTML, рассмотрение HttpOnly cookies.

ЗАКЛЮЧЕНИЕ

Выводы

Поставленная цель достигнута: создано приложение, позволяющее пользователю тренировать печать на русском и английском языках, проходить уроки, выполнять тесты скорости и получать измеримые метрики. Для авторизованных пользователей реализовано сохранение результатов и просмотр агрегированной статистики.

Архитектура проекта соответствует выбранной траектории: backend построен на Django и Django REST Framework, frontend - на React, для real-time сценария использованы Django Channels и WebSocket, а разграничение доступа выполнено через JWT и permissions.

Результаты

Основными результатами являются: реализованная предметная модель Lesson/TestResult/User, REST API для уроков и результатов, JWT-аутентификация с refresh-токенами, интерактивный интерфейс теста и урока, личная статистика, WebSocket-уведомления и backend-тесты ключевых правил.

Проектная документация дополнена PlantUML-диаграммами: контекстом бизнес-процесса, Use Case, доменной моделью, компонентной схемой, ER-диаграммой и диаграммами последовательности.

Перспективы развития

Дальнейшее развитие проекта может включать расширение библиотеки уроков, адаптивную генерацию упражнений на основе ошибок пользователя, рейтинги, достижения, экспорт статистики, полноценную административную панель во frontend и OpenAPI-документацию.

Для промышленной эксплуатации рекомендуется перейти на PostgreSQL и Redis, настроить HTTPS, мониторинг, резервное копирование, CI/CD, расширить автоматические frontend-тесты и усилить защиту токенов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Кузьмин, А. Г. Разработка фронтенд-приложений / А. Г. Кузьмин. — Санкт-Петербург : Питер, 2025. — 496 с. — (Библиотека программиста). — ISBN 978-5-4461-4272-9.
2. Кухтин, С. А. Frontend-разработка сайтов и приложений на HTML, CSS, JavaScript и React / С. А. Кухтин. — Санкт-Петербург : Наука и Техника, 2026. — 512 с. — ISBN 978-5-907592-84-1.
3. Орбайсета, А. С. Создание фронтенд-фреймворка с нуля / А. С. Орбайсета. — Санкт-Петербург : Питер, 2025. — 384 с. — (Библиотека программиста). — ISBN 978-5-4461-4201-9.
4. Курс «Фронтенд-разработчик» от Яндекс Практикум. — URL: <https://practicum.yandex.ru/frontend-developer/> (дата обращения: 10.03.2026).
5. Проектирование и разработка WEB-приложений. Введение в frontend и backend разработку на JavaScript и node.js : учебное пособие для вузов. — 4-е изд., стер. — Санкт-Петербург : Лань, 2025. — 120 с. — ISBN 978-5-507-51011-5.
6. Чернышев, С. А. Основы Flutter / С. А. Чернышев, Ю. М. Петров, С. П. Ильин, П. А. Гершевич. — Санкт-Петербург : Питер, 2026. — 688 с. — (Библиотека программиста). — ISBN 978-5-4461-4469-3.
7. Дронов, В. А. Django 4: практика создания веб-сайтов / В. А. Дронов. — Санкт-Петербург : БХВ-Петербург, 2024. — 544 с. — ISBN 978-5-9775-6892-6.
8. Ломов, А. Ю. Веб-разработка на Django 5 / А. Ю. Ломов. — Москва : ДМК Пресс, 2025. — 412 с. — ISBN 978-5-97060-978-1.
9. Документация Django REST Framework. — URL: <https://www.django-rest-framework.org/> (дата обращения: 15.03.2026).
10. Sanders, D. Django REST Framework Simple JWT Documentation. — Version 5.5.1. — URL: <https://django-rest-framework-simplejwt.readthedocs.io/> (дата обращения: 15.03.2026).
11. Документация Django Channels. — URL: <https://channels.readthedocs.io/> (дата обращения: 18.03.2026).

12. Django REST Framework Documentation — Serializers. — URL: <https://www.django-rest-framework.org/api-guide/serializers/> (дата обращения: 15.03.2026).
13. Project Stage Academy. Notification System for New Messages (GitHub Issue #73). — 2025. — URL: https://github.com/Project-Stage-Academy/ChNU_PM_2025/issues/73 (дата обращения: 25.03.2026).
14. Zoma00. Django Notifications: Real-Time Notifications with WebSockets. — GitHub Repository. — 2025. — URL: <https://github.com/zoma00/Django-notifications> (дата обращения: 08.04.2026).
15. Документация TanStack React Query. — URL: <https://tanstack.com/query/latest> (дата обращения: 23.04.2026).
16. Документация TanStack React Query: Optimistic Updates. — URL: <https://tanstack.com/query/v5/docs/framework/react/guides/optimistic-updates> (дата обращения: 23.04.2026).
17. Документация Axios. — URL: <https://axios-http.com/> (дата обращения: 25.04.2026).
18. Документация React Router. — URL: <https://reactrouter.com/> (дата обращения: 20.04.2026).
19. Документация PostgreSQL. — URL: <https://www.postgresql.org/docs/> (дата обращения: 15.05.2026).
20. Документация Redis. — URL: <https://redis.io/documentation> (дата обращения: 15.05.2026).
21. Документация PlantUML. — URL: <https://plantuml.com/> (дата обращения: 14.05.2026).
22. Henderson, J. P. PlantUML Examples for UML, ERD, Gantt Charts and More. — GitHub Repository. — URL: <https://github.com/joelparkerhenderson/plantuml-examples> (дата обращения: 28.04.2026).

23. MDN Web Docs. WebSocket API. — URL: <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket> (дата обращения: 15.04.2026).

24. React Documentation — Hooks: useContext, useReducer. — URL: <https://react.dev/reference/react/useContext> (дата обращения: 20.04.2026).

25. Официальная документация Python для Django. — URL: <https://docs.djangoproject.com/> (дата обращения: 15.03.2026).

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

СКРИНШОТЫ ИНТЕРФЕЙСОВ

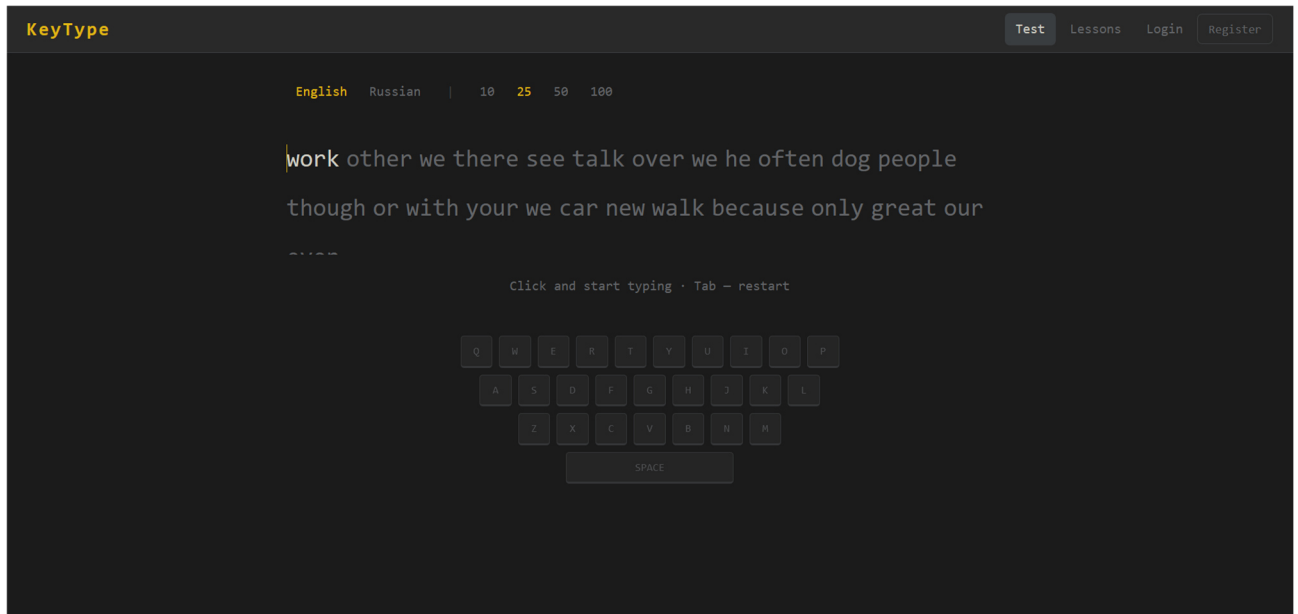


Рисунок 1 – Страница теста

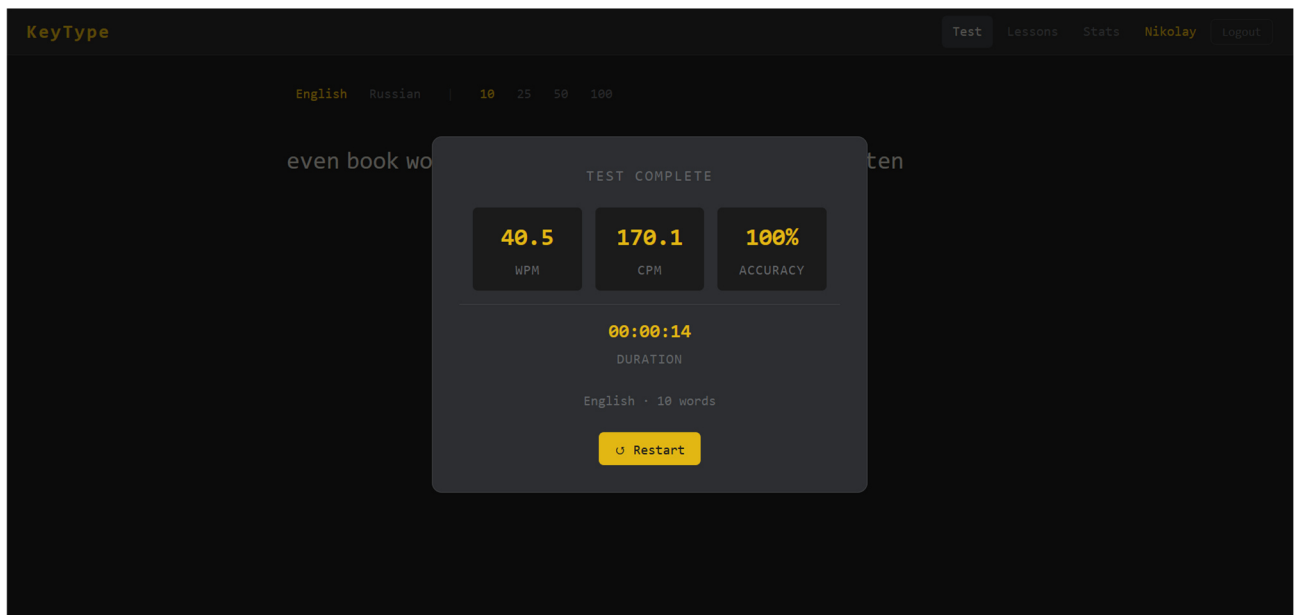


Рисунок 2 – Модальное окно со статистикой теста

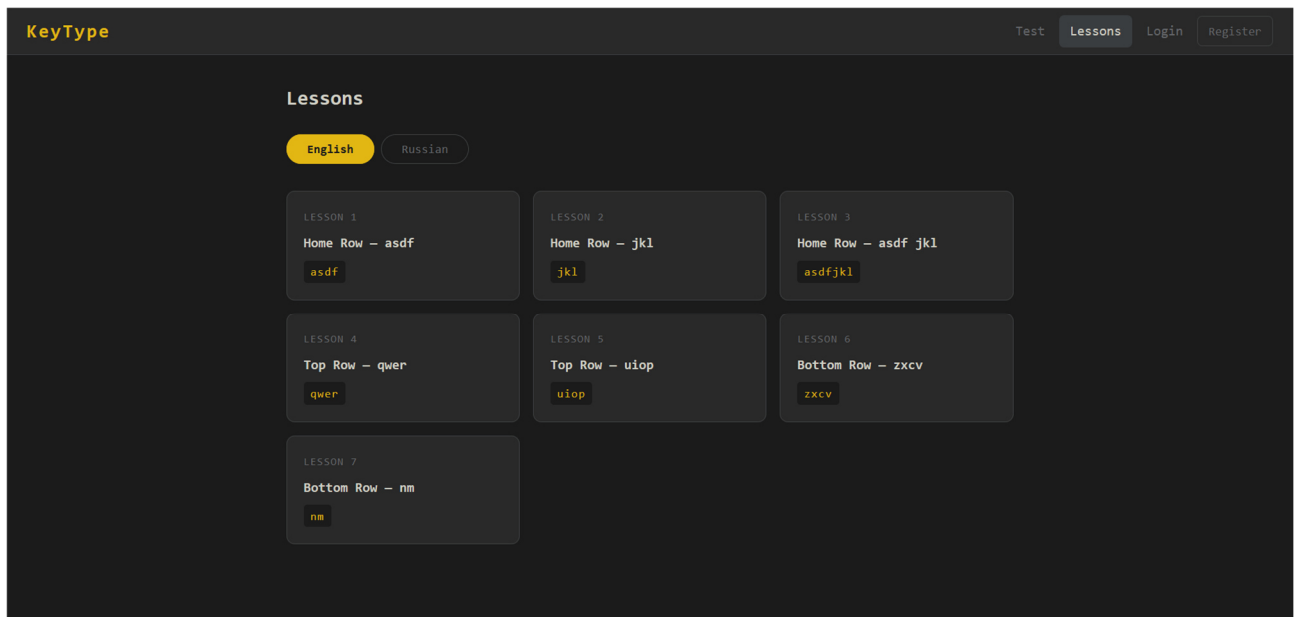


Рисунок 3 – Страница списка уроков

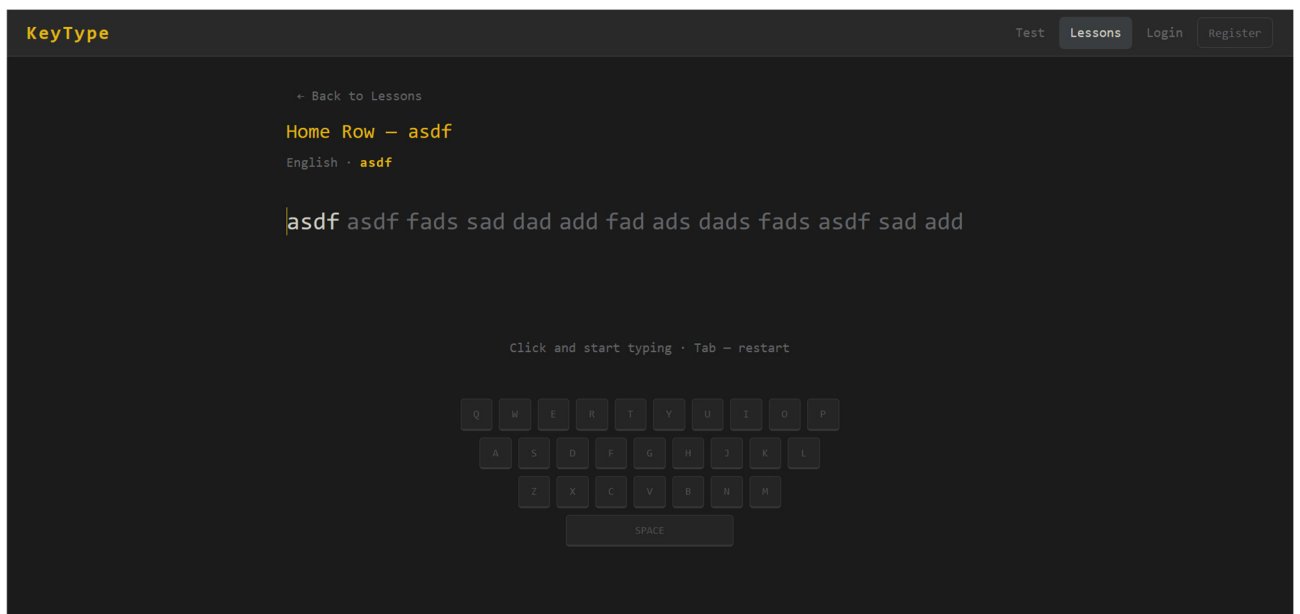


Рисунок 4 – Страница урока

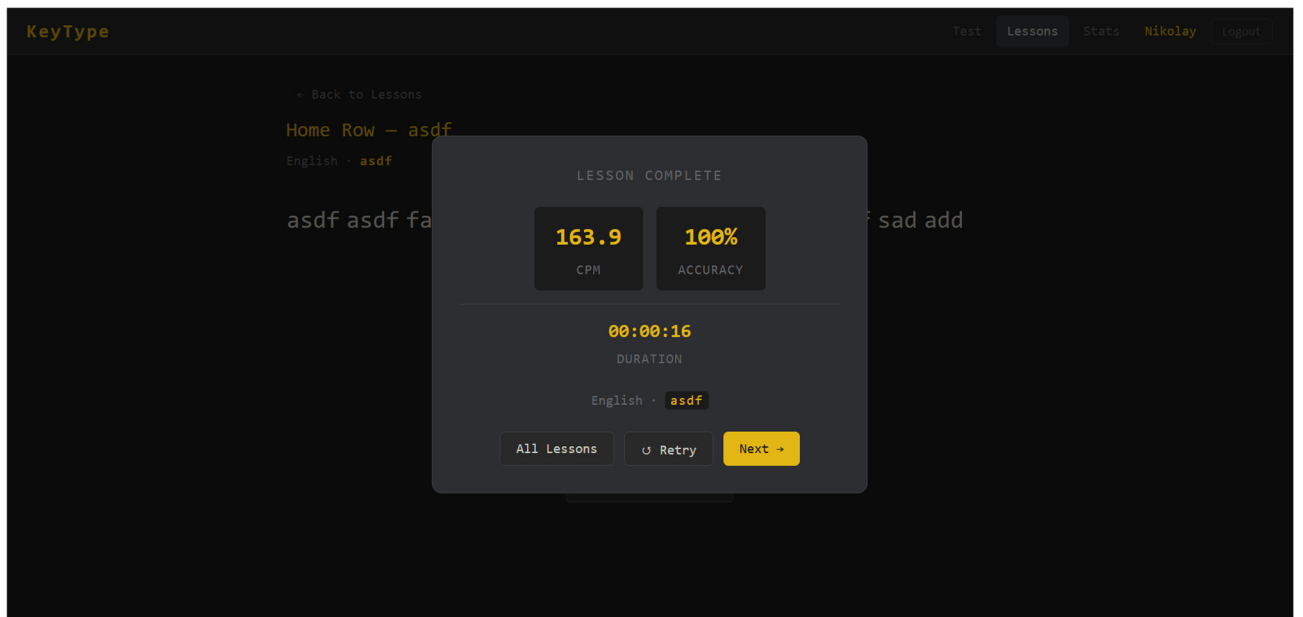


Рисунок 5 – Модальное окно со статистикой урока

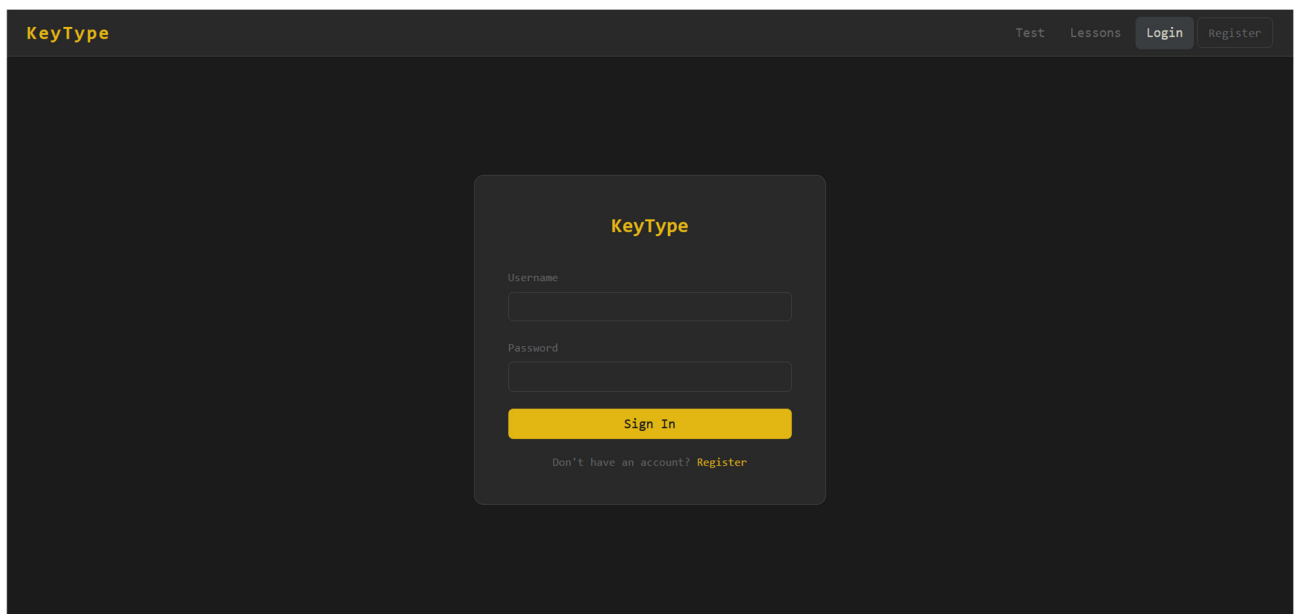


Рисунок 6 – Страница входа

KeyType

Test Lessons Login Register

Create Account

Username

Email

Password

Confirm Password

Register

Already have an account? [Sign in](#)

Рисунок 7 – Страница регистрации

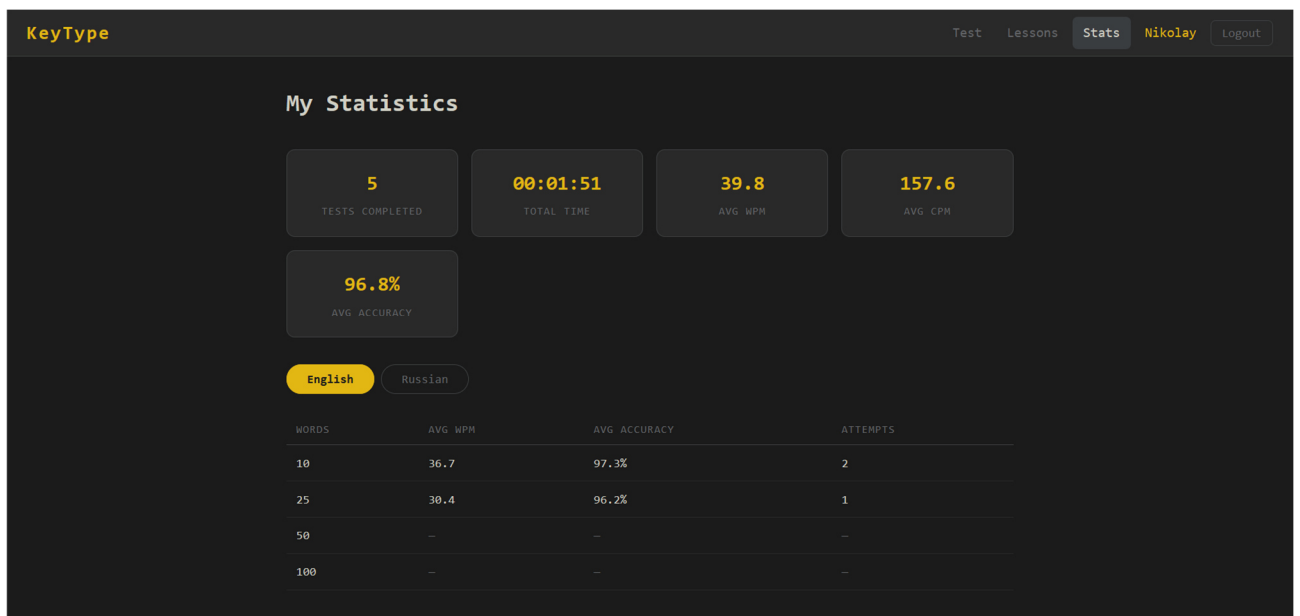


Рисунок 8 – Страница общей статистики

ПАСПОРТ ПРОЕКТА

Веб-приложение на Django + React

Параметр	Значение
Название проекта	KeyType – веб-приложение клавиатурного онлайн-тренажёра
Тип проекта	Веб-приложение (Django REST Framework + React SPA)
Версия платформы	Django 5.2 / Python 3.12, React 19
Язык разработки	Python, JavaScript (ES6+)
Руководитель проекта	Ф. В. Самойлов, доцент межинститутской базовой кафедры

Цель проекта: разработать полнофункциональное веб-приложение для тренировки скорости и точности клавиатурной печати с разделением на клиентскую и серверную части, JWT-аутентификацией и уведомлениями в реальном времени.

Ключевой функционал

№	Функция	Компонент
1	Тест	TestPage → useTypingTest → core.views
2	Уроки	LessonsPage → LessonPracticePage → core.views
3	Статистика	StatsPage → core.views.user_stats
4	Авторизация и регистрация	Entity Framework Core + SQL Server
5	Админ-панель	Django Admin → core.admin.LessonAdmin

Технологический стек

Backend	Frontend	Инфраструктура
Django 5.2, Django REST Framework 3.17, Django Channels 4.3, Simple JWT 5.5, drf-spectacular	React 19, React Router 7, Axios 1.16, React Query 5, WebSocket API	Python 3.12, Node.js / npm, PostgreSQL / SQLite, Daphne (ASGI)